

CNNs and UNets

Carlos Cotrini and Winston Rohmann
Building ML/AI applications
2026



Four main tasks in computer vision

Regression and
classification of images



Cat? Yes

Dog? Yes

Bat? No

Feature extraction



Cat color: black

Dog color: brown

Cat's eye color:
green

Dog's eye color:
brown

Segmentation and
detection

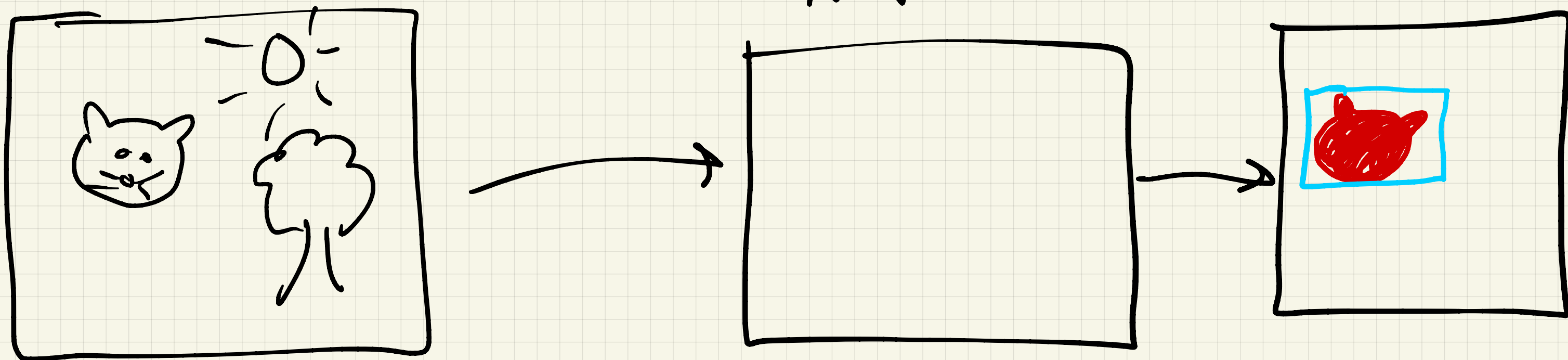


Generation



The problem of Segmentation & recognition

NN



Before

Now

$\hat{y}$ is a number
 $\hat{y}$ is an image

Strategy

2) Model

NN

$\nabla \bar{l}$

1) Data

y

x

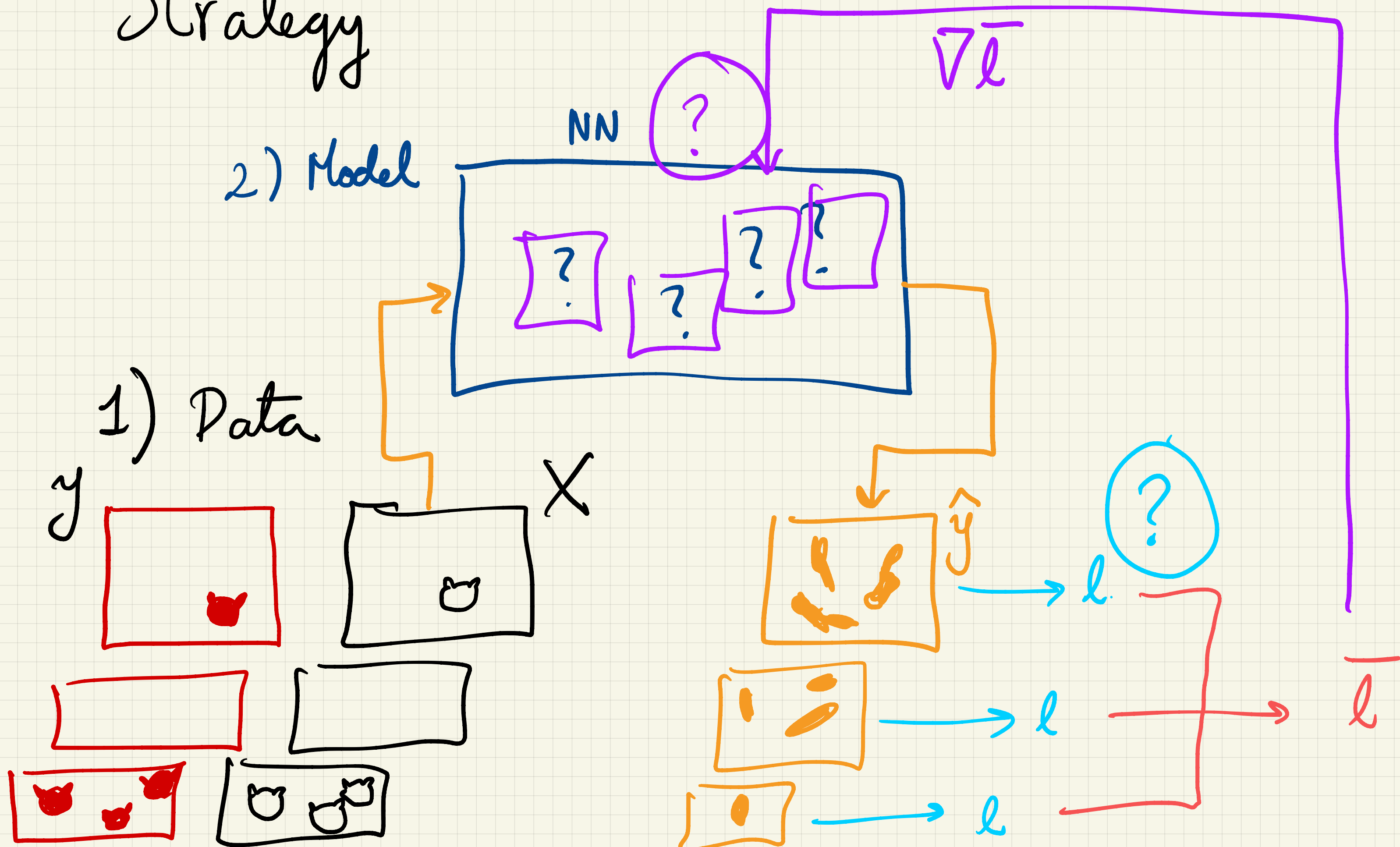
$\hat{y}$

l

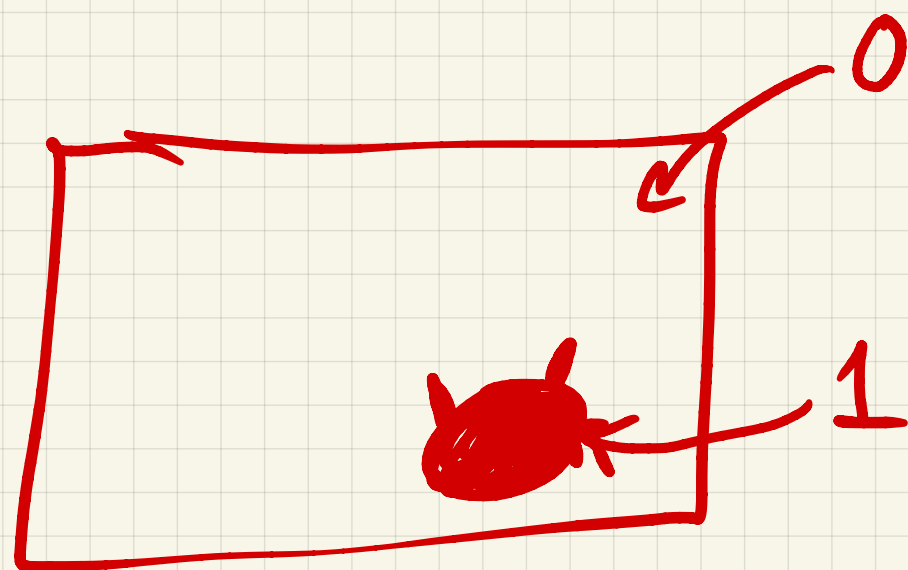
l

l

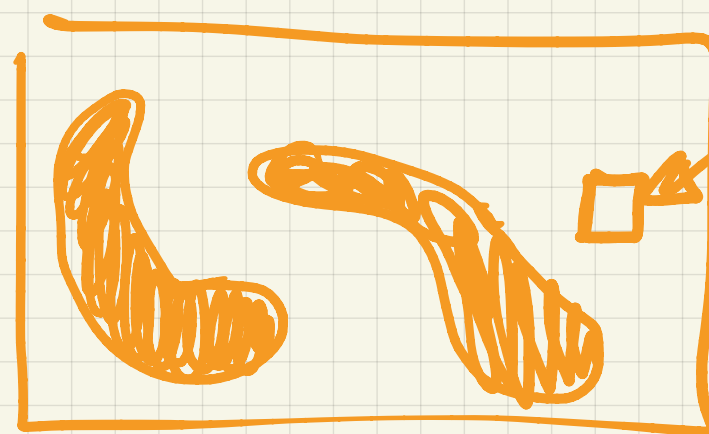
$\bar{l}$



How to do l ?



y



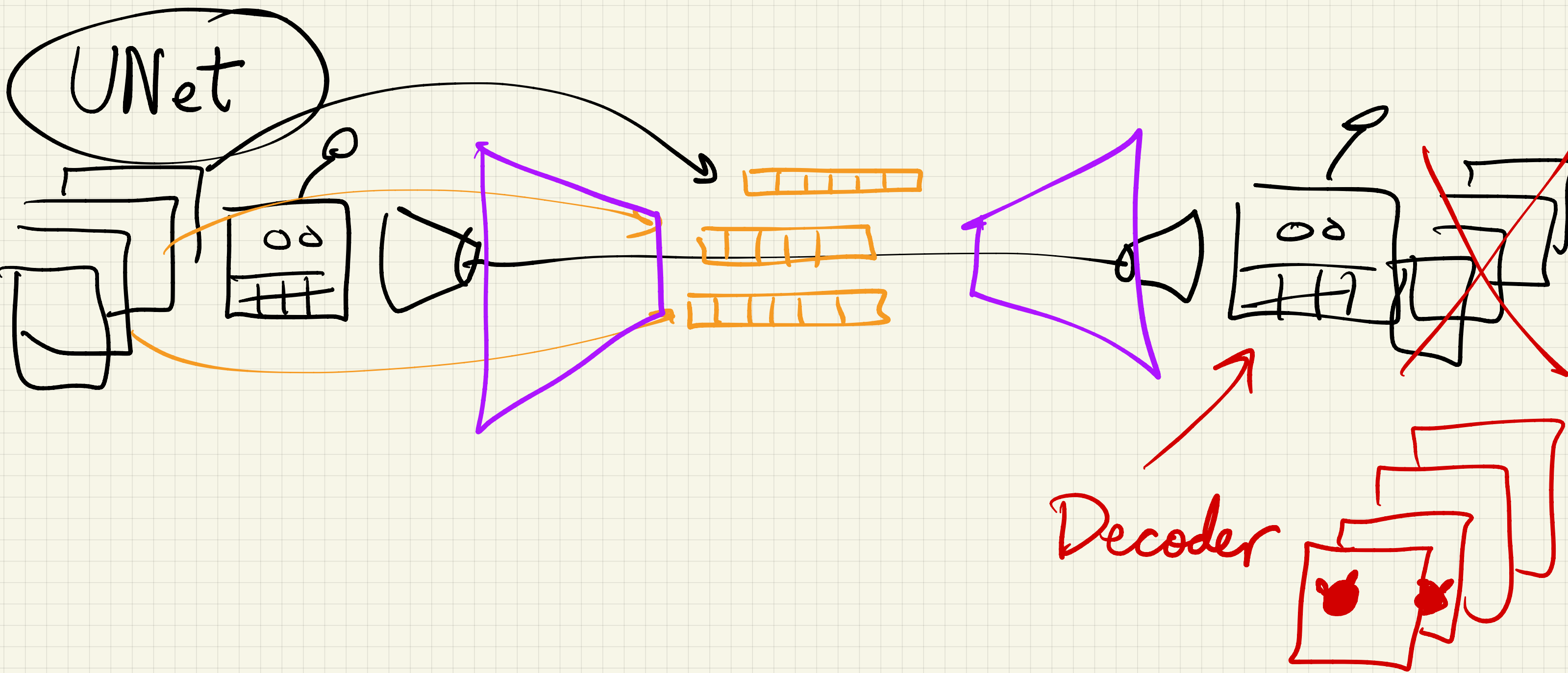
prob that this is
a pixel of a cat

y

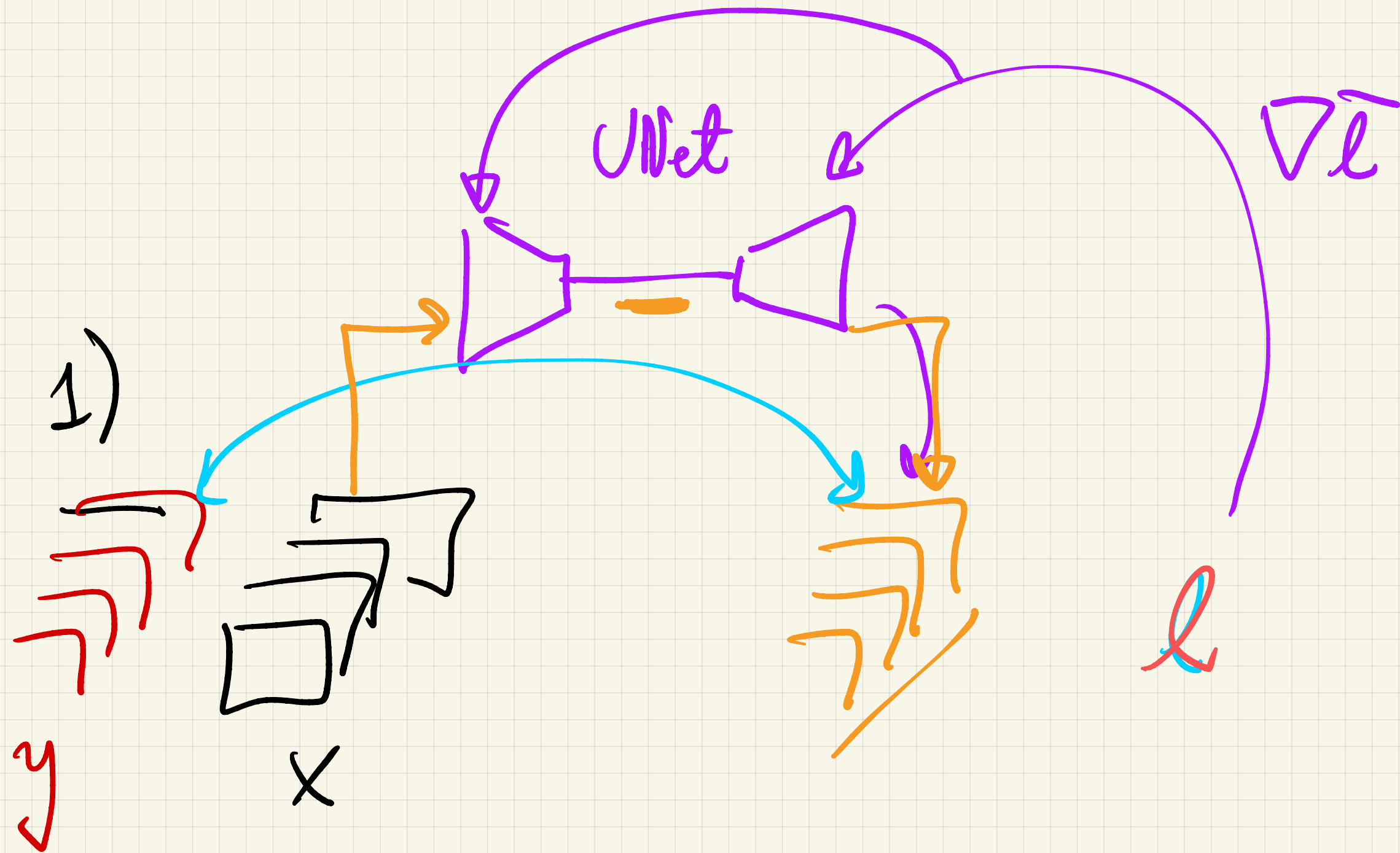
1st attempt

$l = \text{sum over all pixels}$

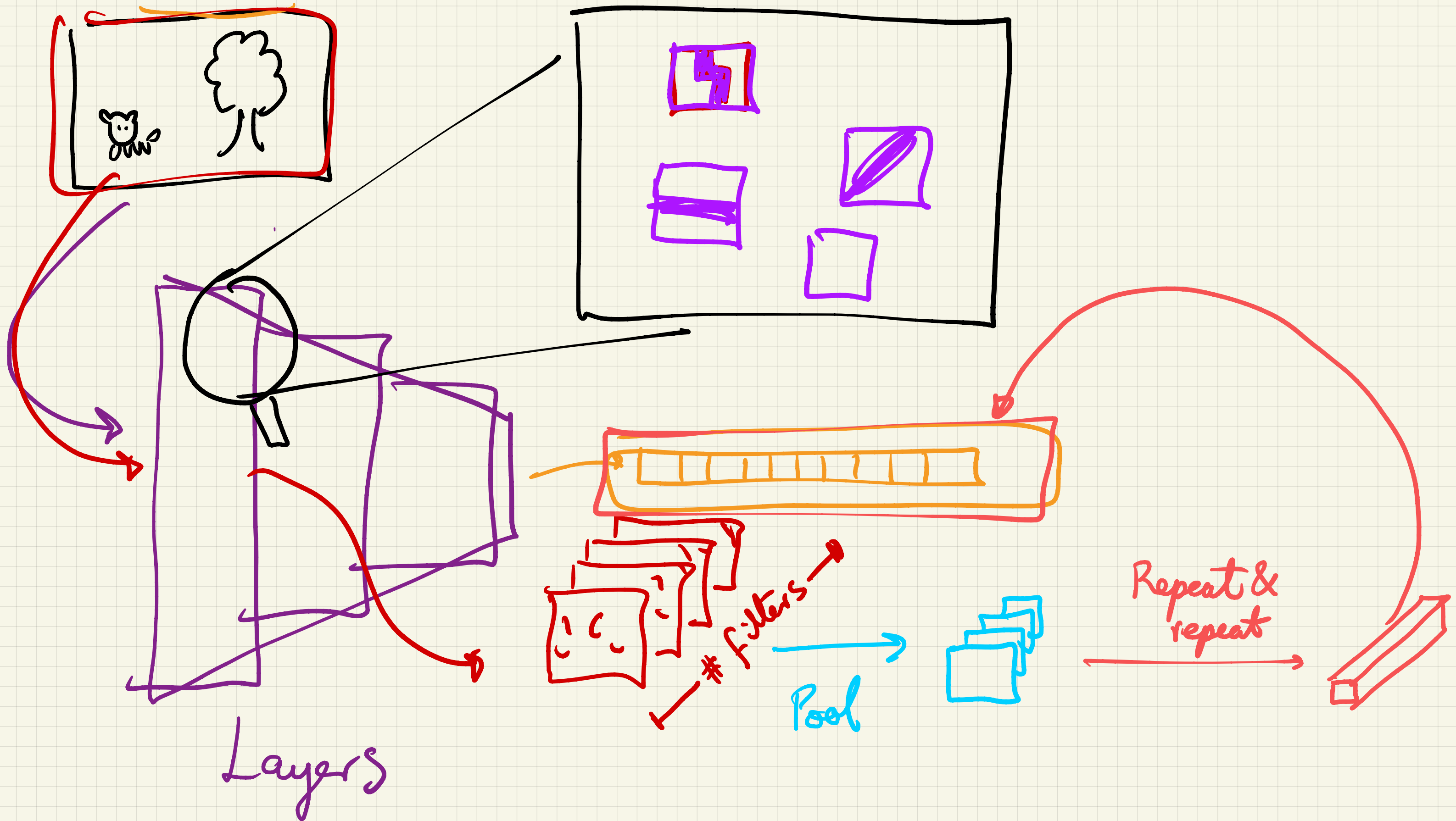
$\left(\begin{array}{l} \text{pixel at row } i \text{ col } j \\ \text{pixel at row } i \text{ col } j \end{array} - \right)^2$



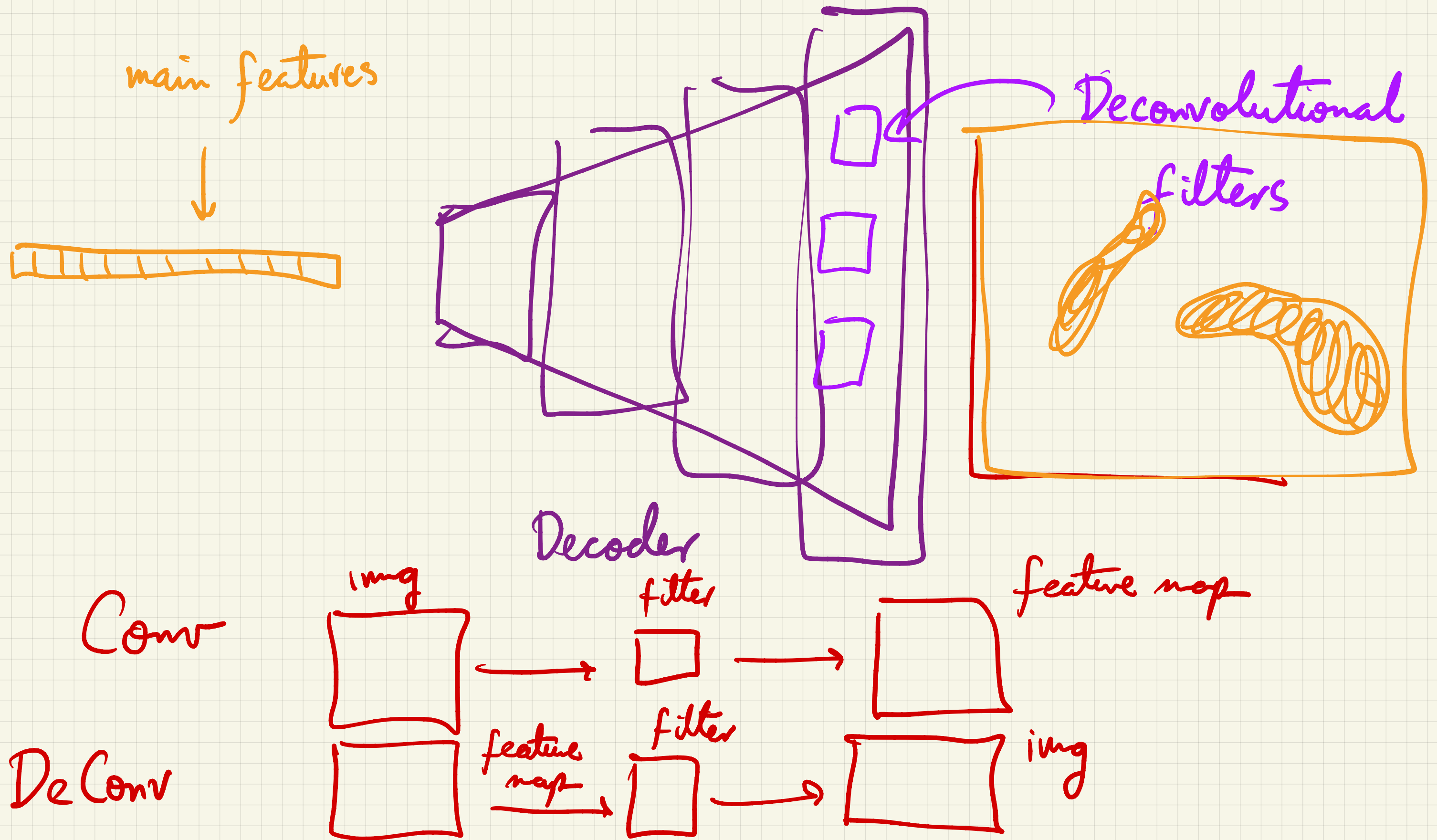
How to ensure that the UNet segments
cats and not trees or dogs?



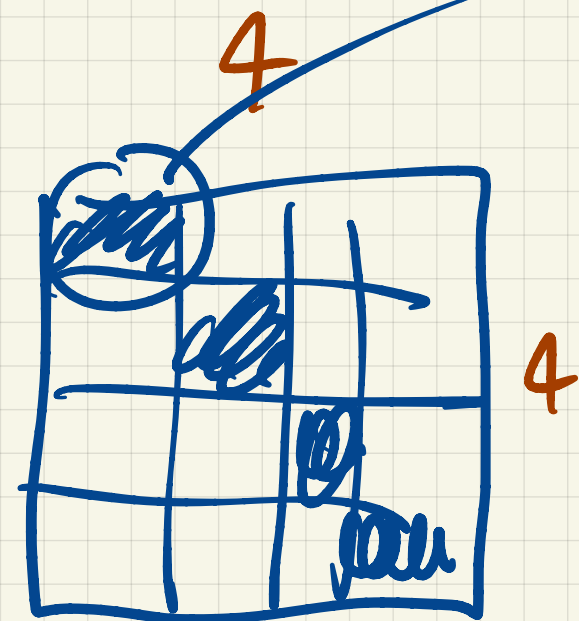
NN (Encoder part)



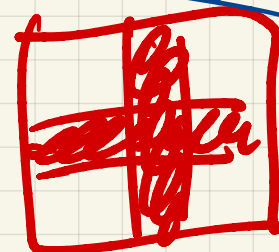
NN (Decoder part)



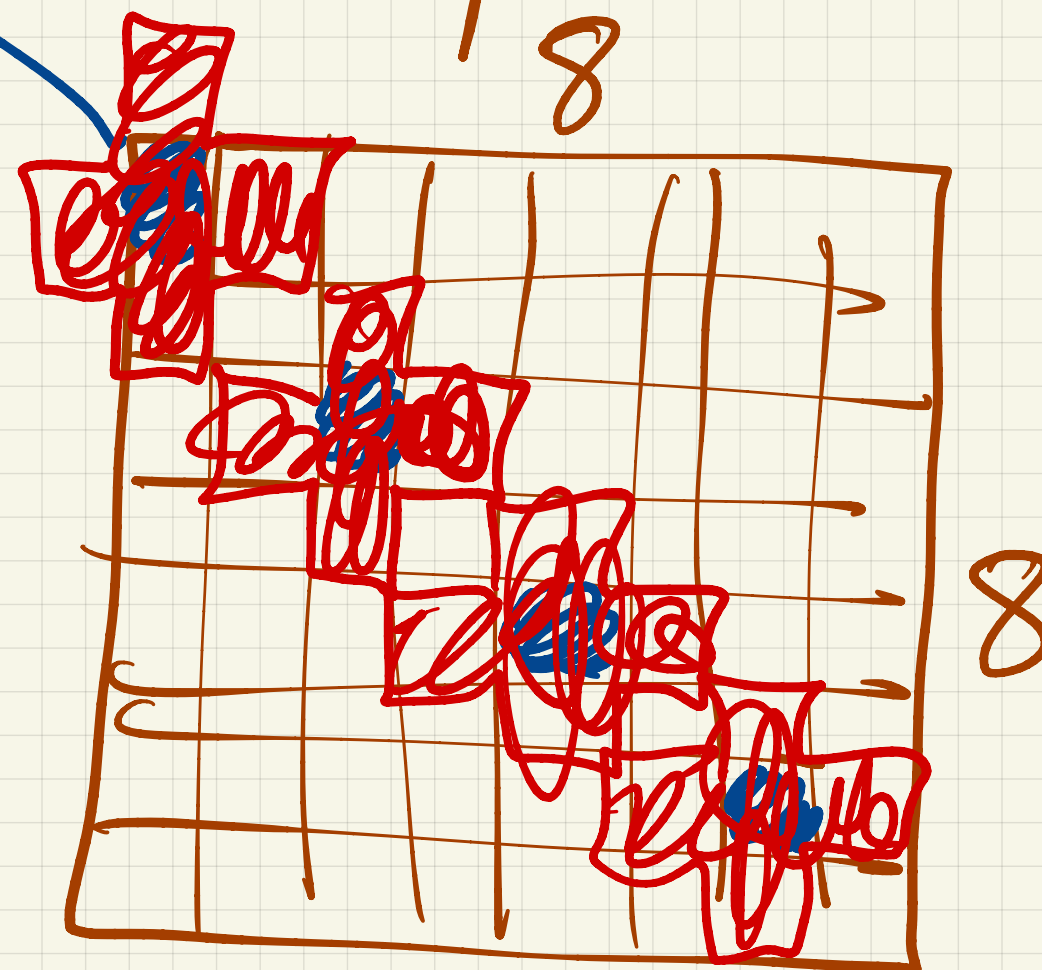
The "de"-pooling
filter



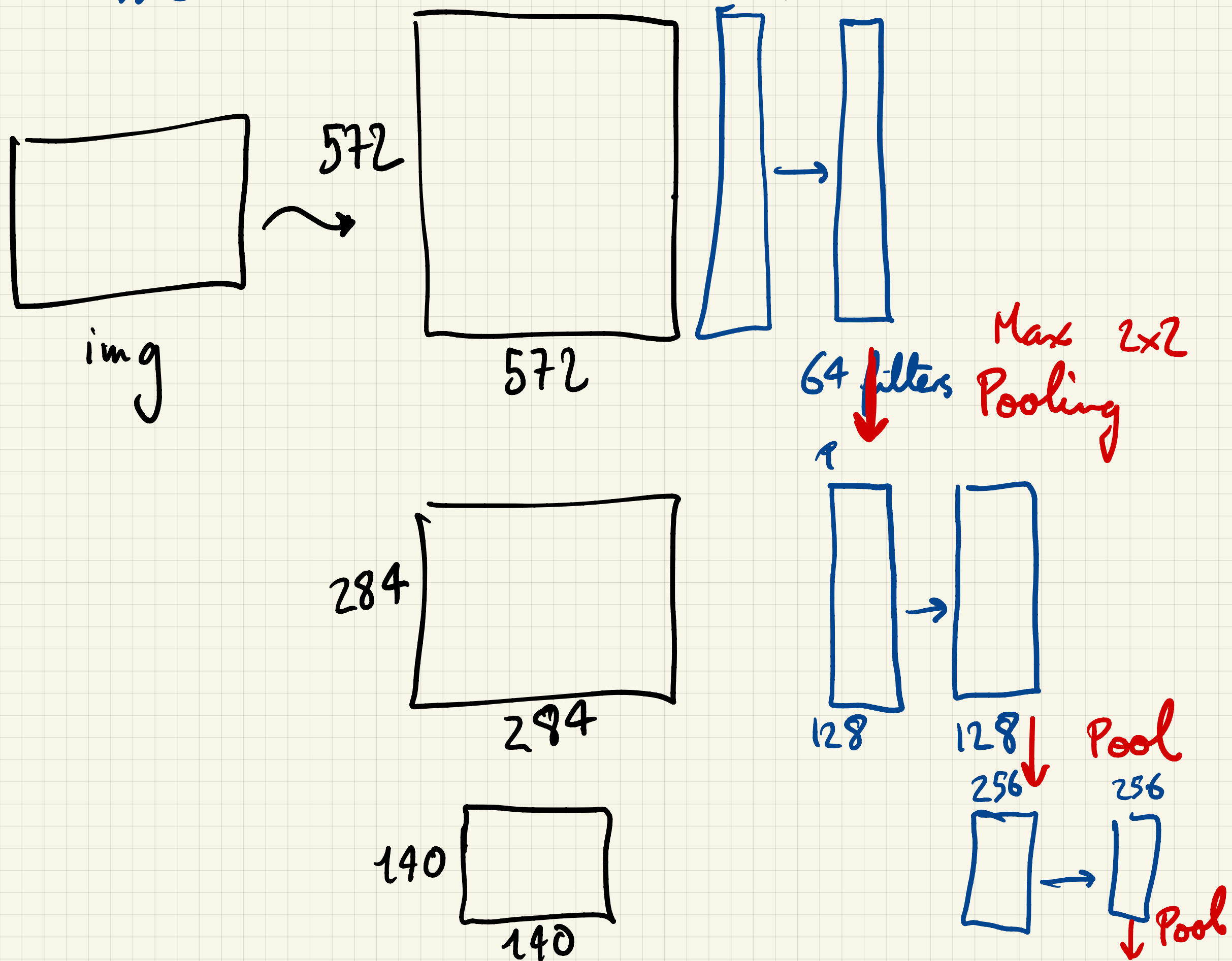
Feat map



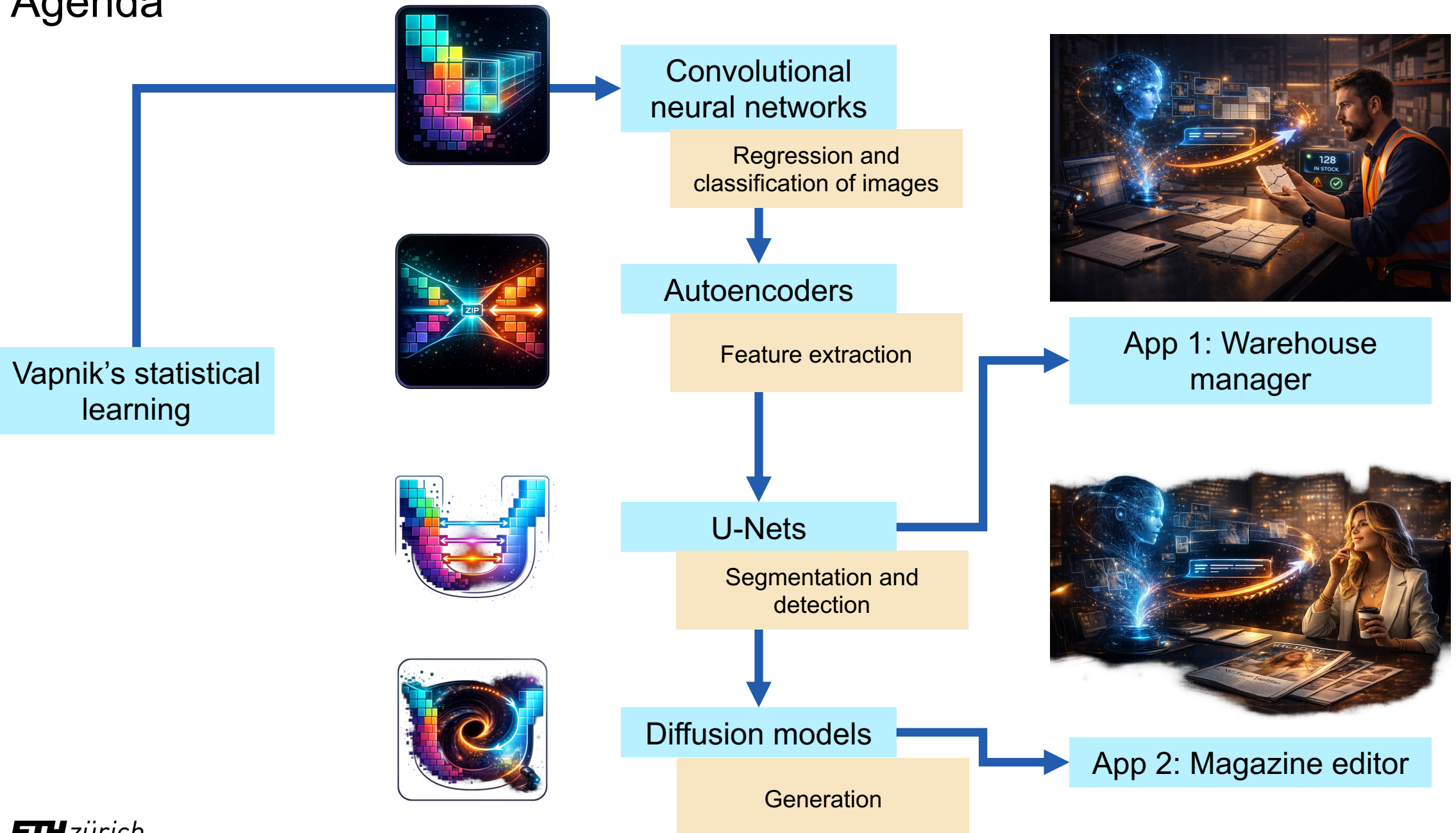
Output
8



The UNet

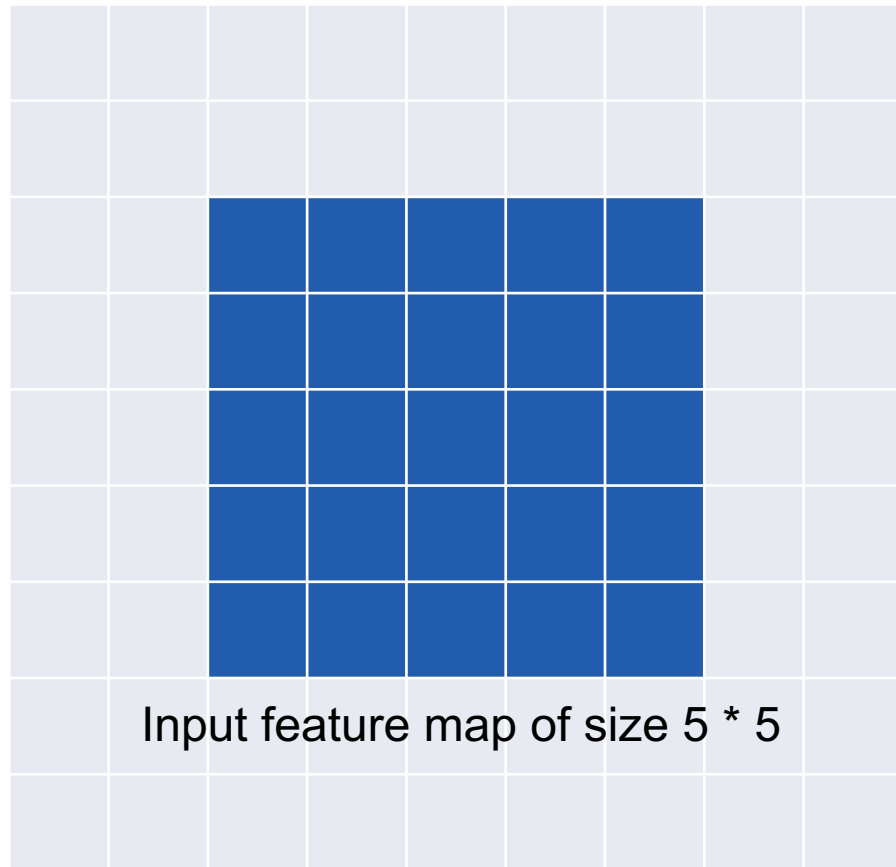


Agenda

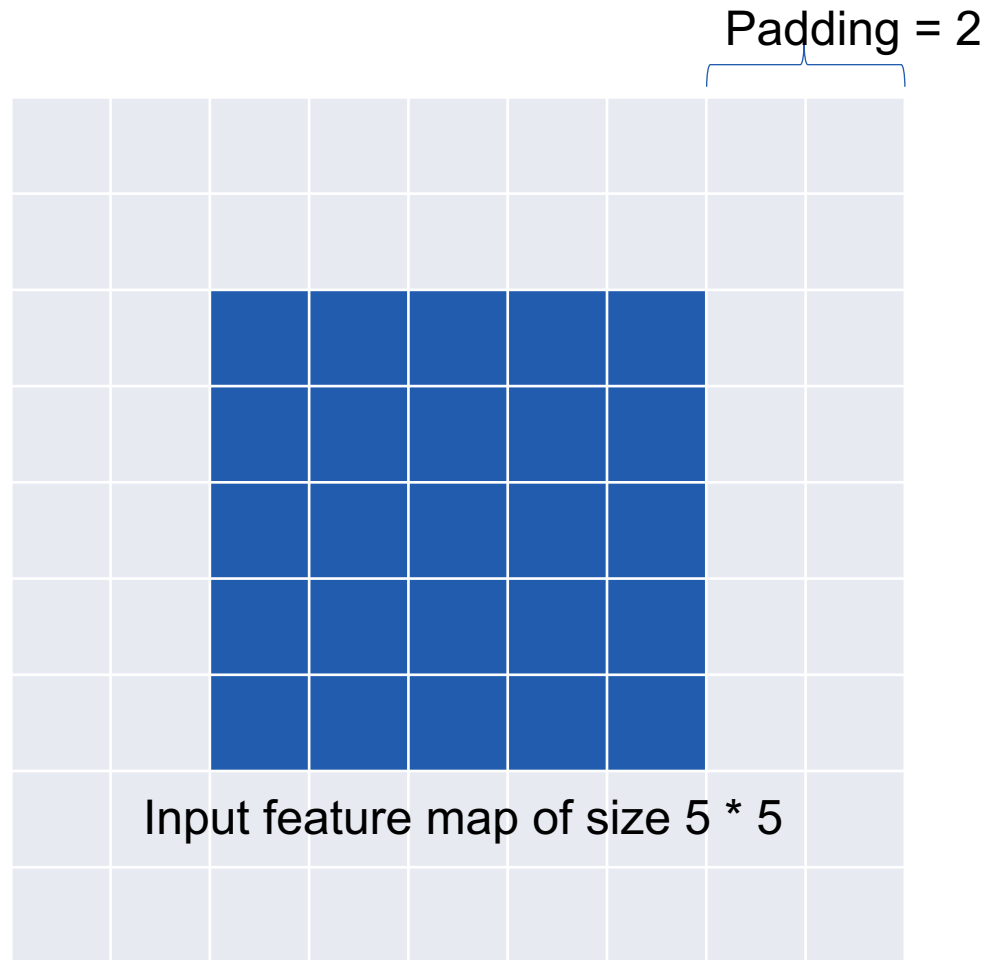


Convolutions and deconvolutions

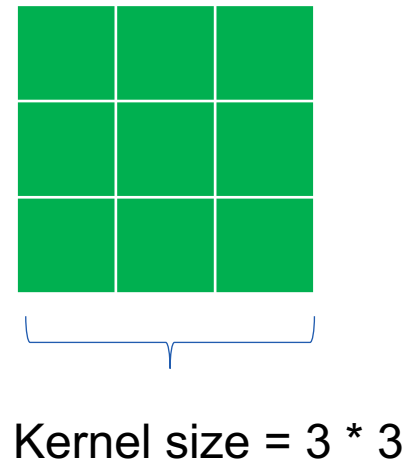
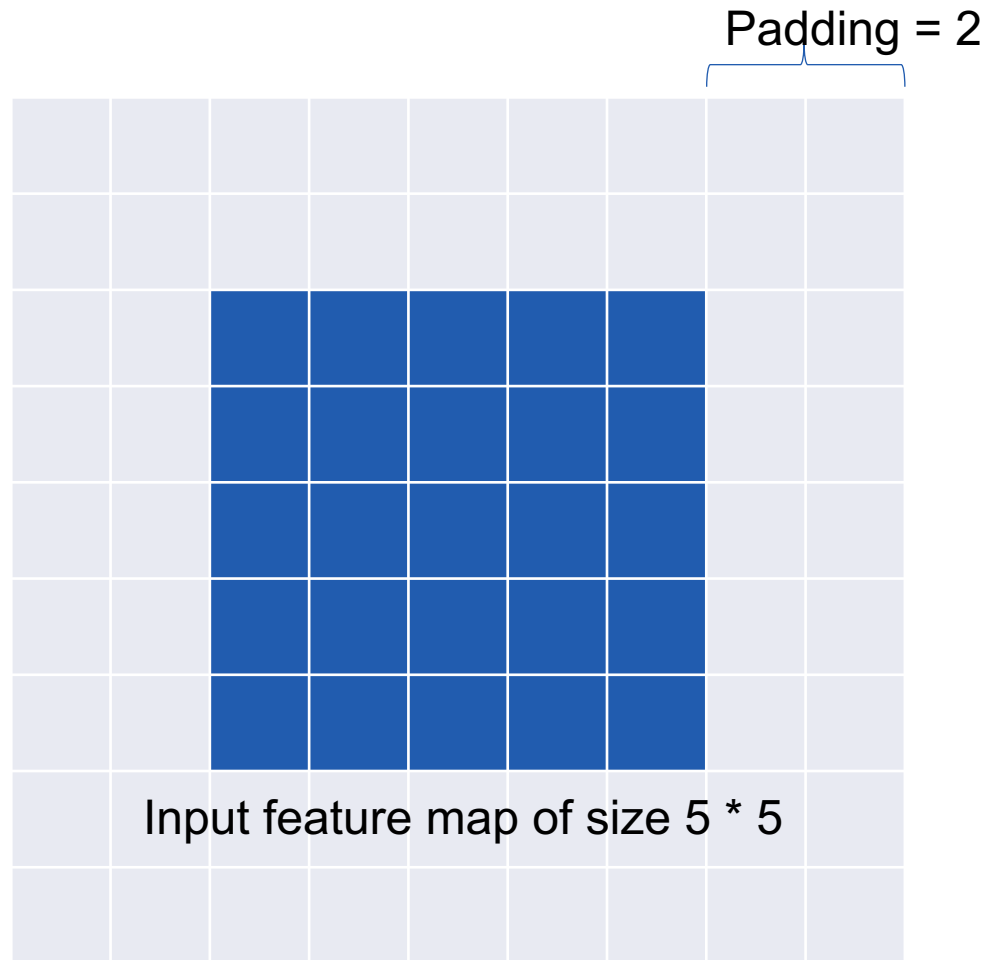
Convolutional filters



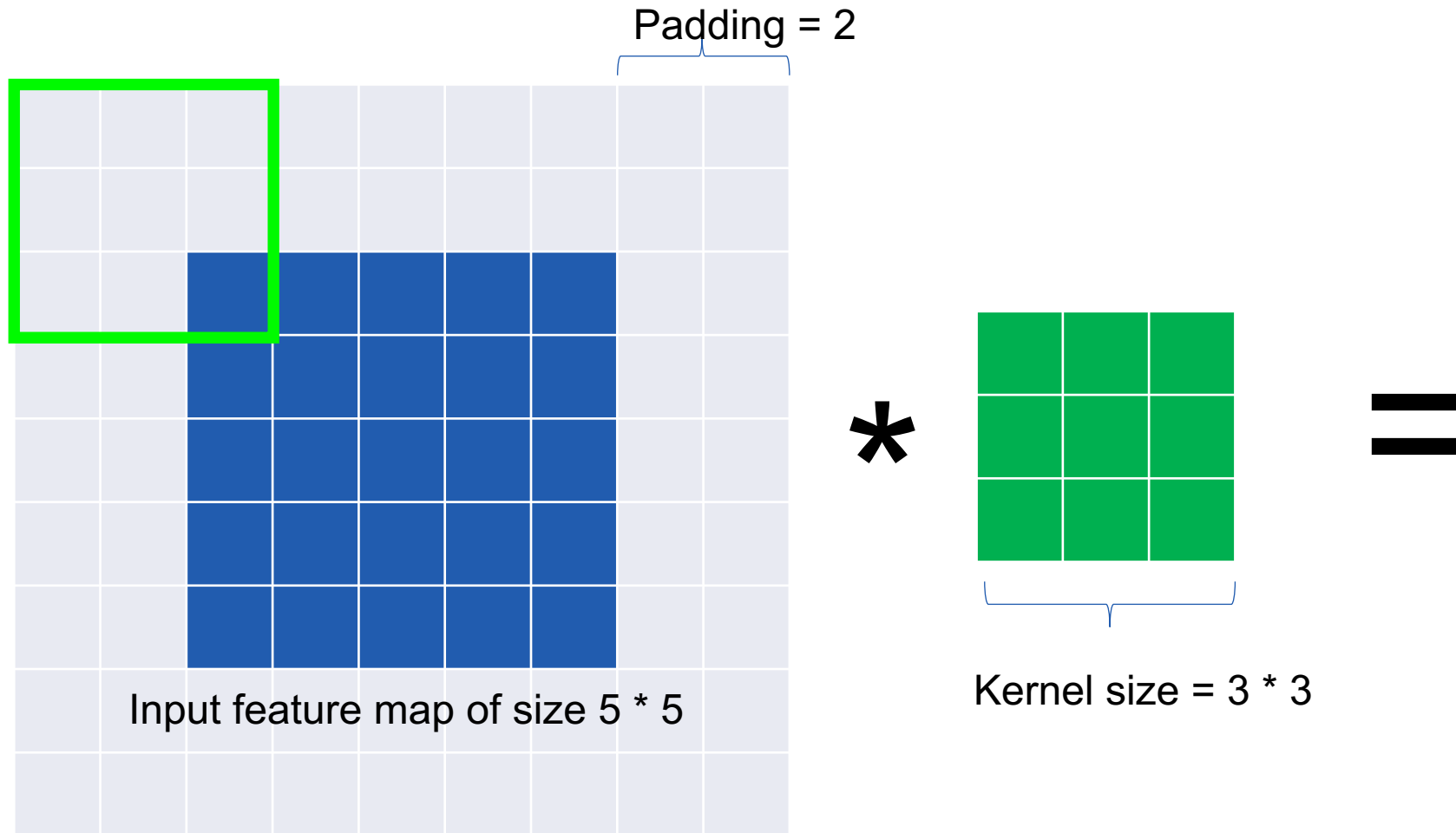
Convolutional filters



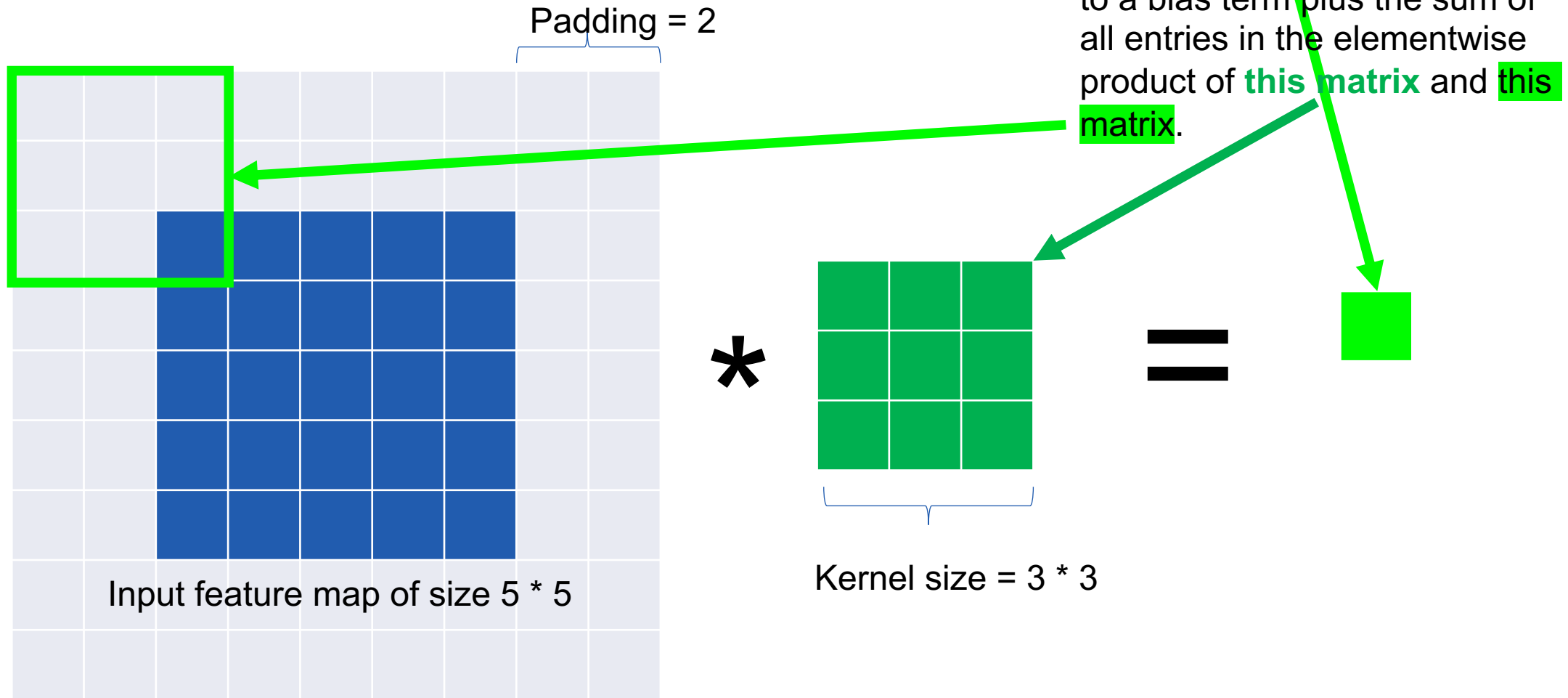
Convolutional filters



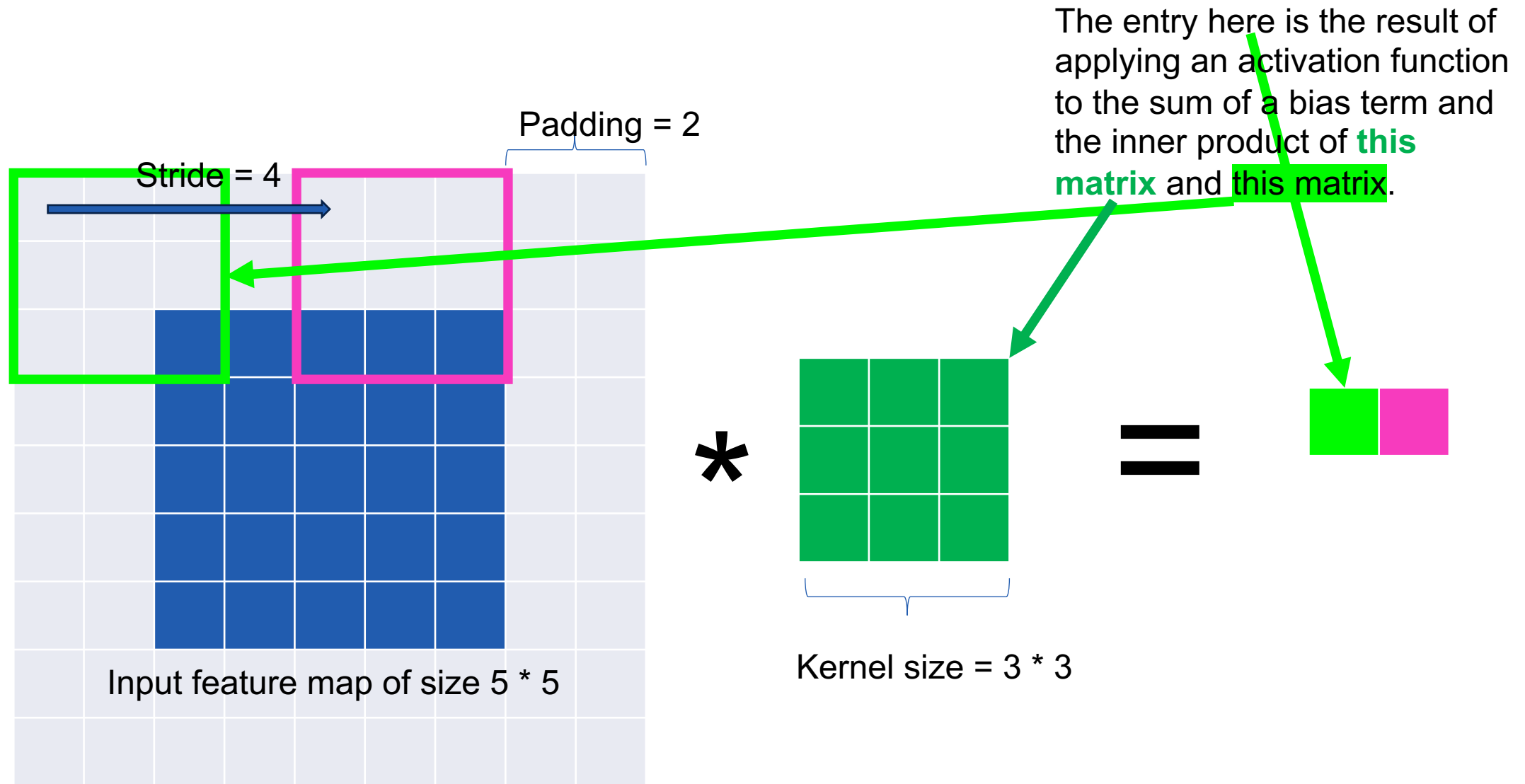
Convolutional filters



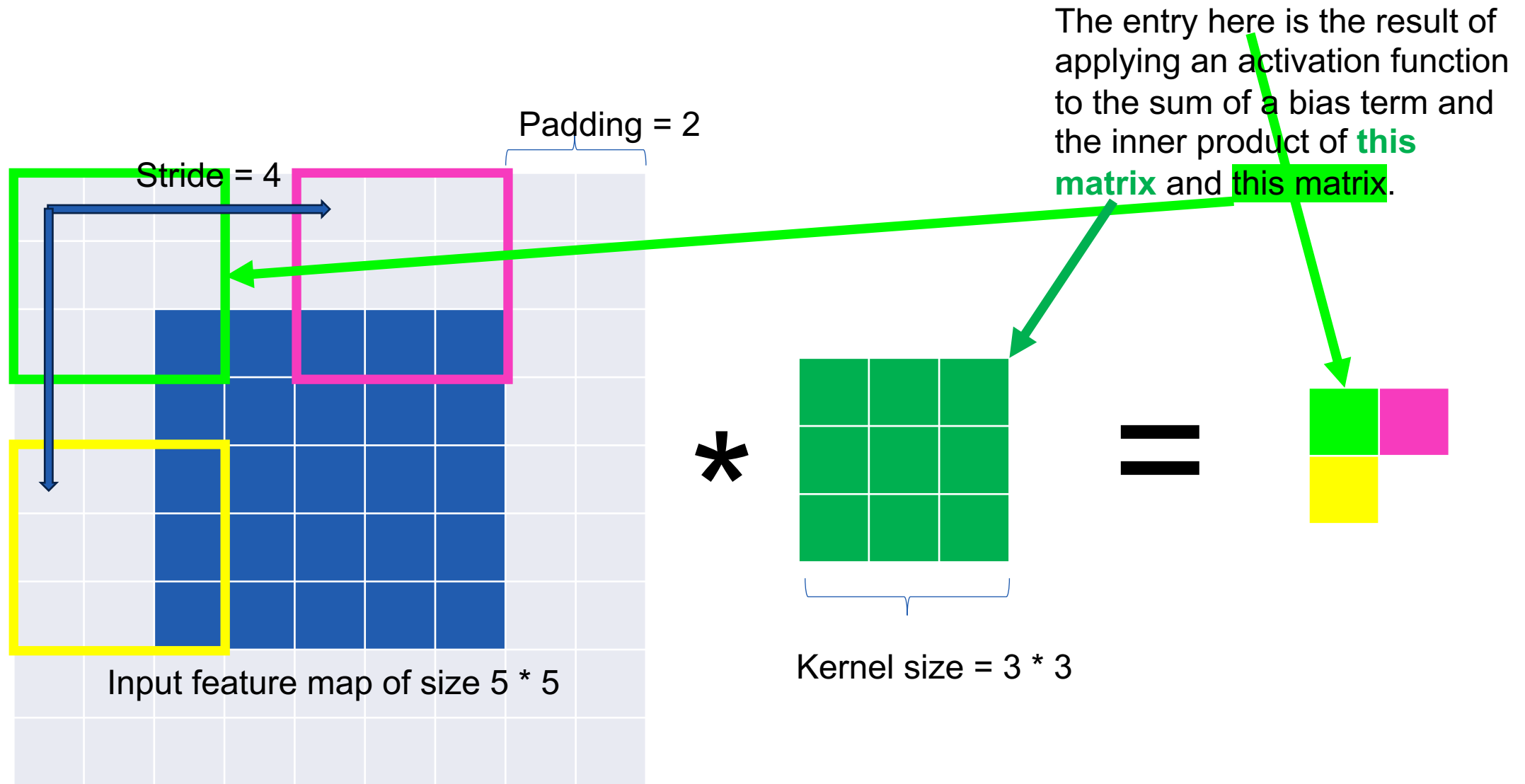
Convolutional filters



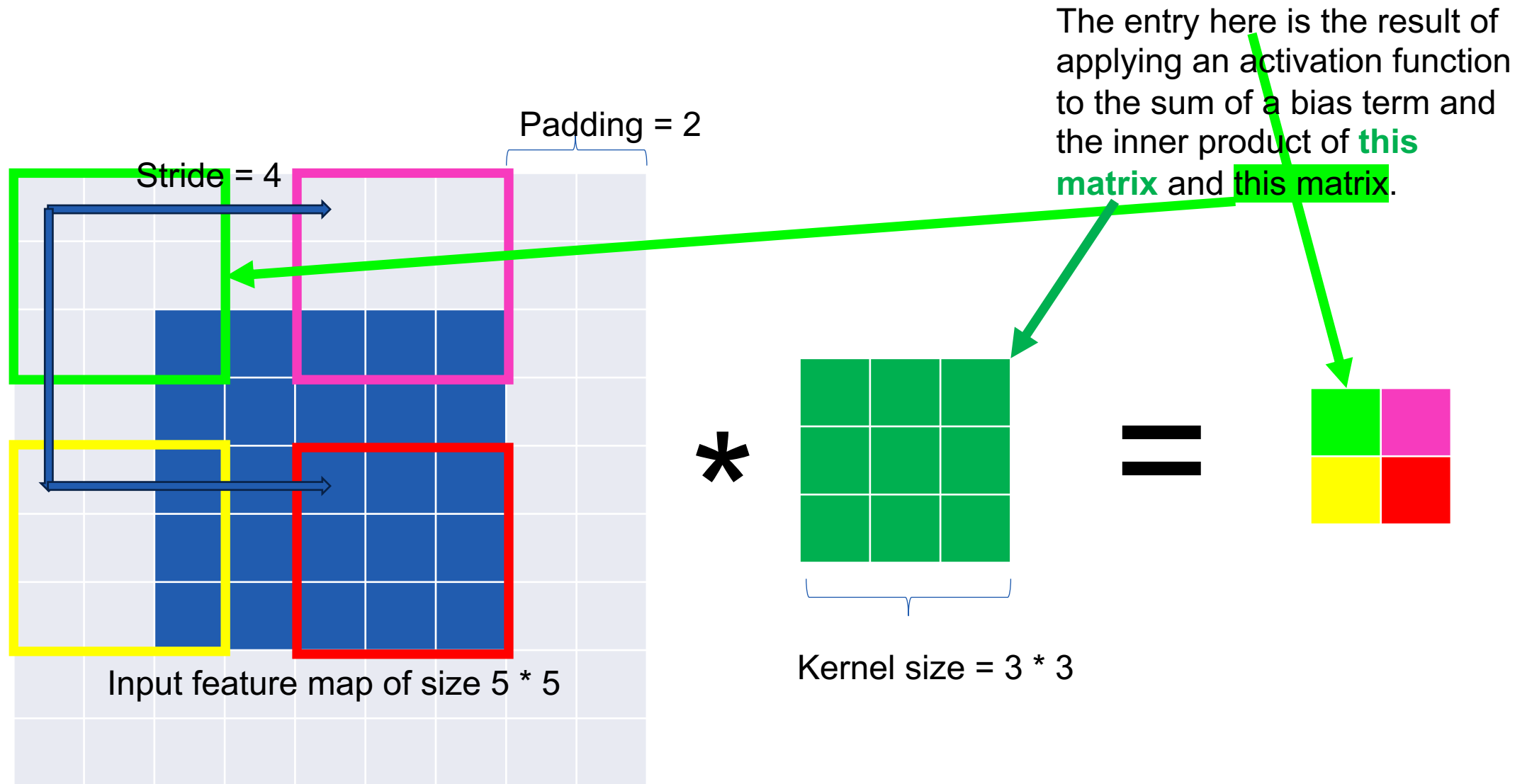
Convolutional filters



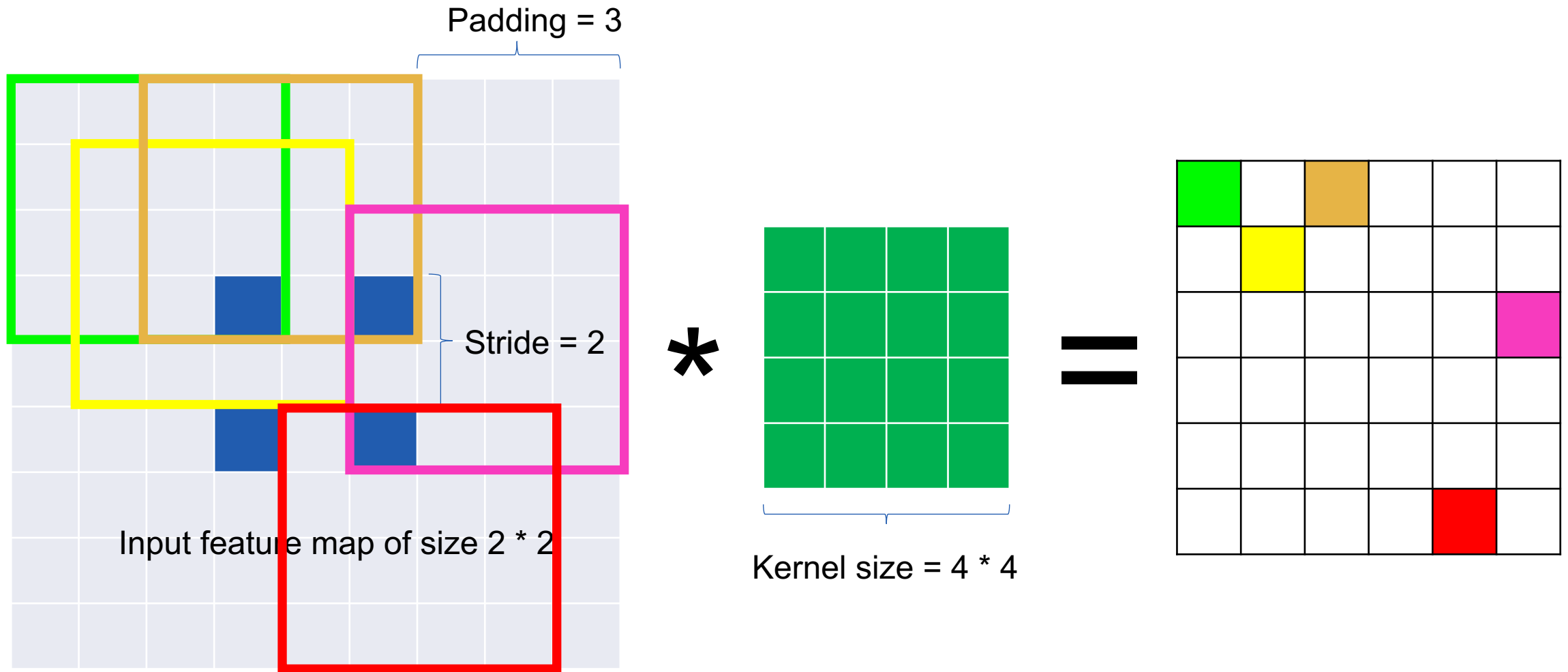
Convolutional filters



Convolutional filters

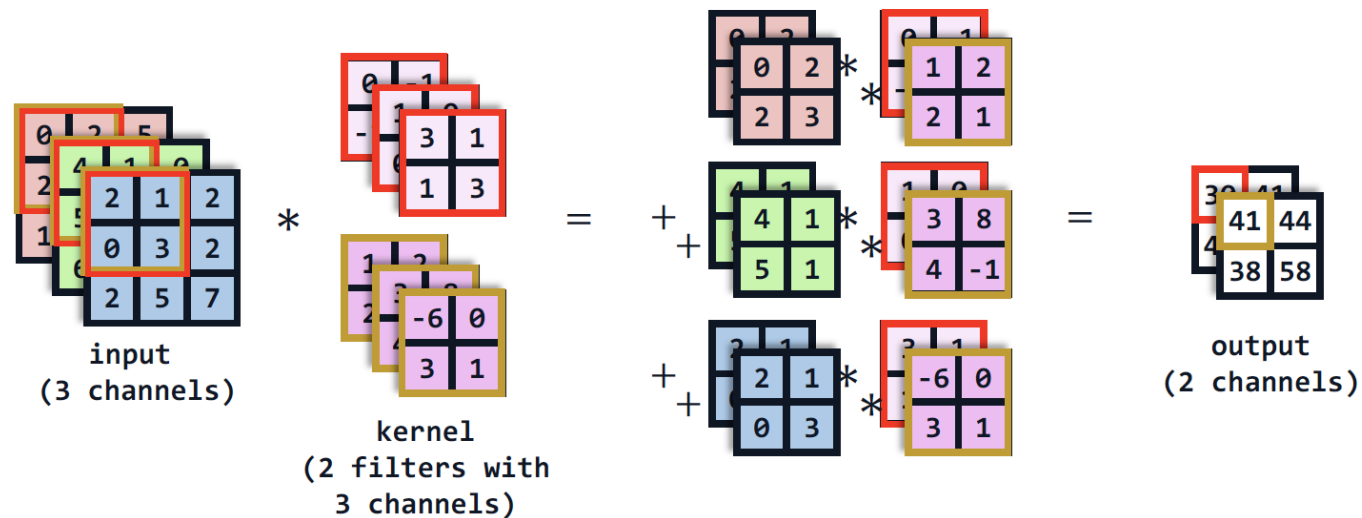


Deconvolutional filters



Convolutional and deconvolutional layers

- A (de)convolutional layer consists of a number of equally-sized (de)convolutional filters, a padding, and a stride.
- The output is a tensor where the number of channels matches the number of filters in the layer.



U-net

- A typical architecture for image segmentation.
- One can also treat a U-net as some sort of image autoencoder
- Image from original paper:
<https://arxiv.org/abs/1505.04597>

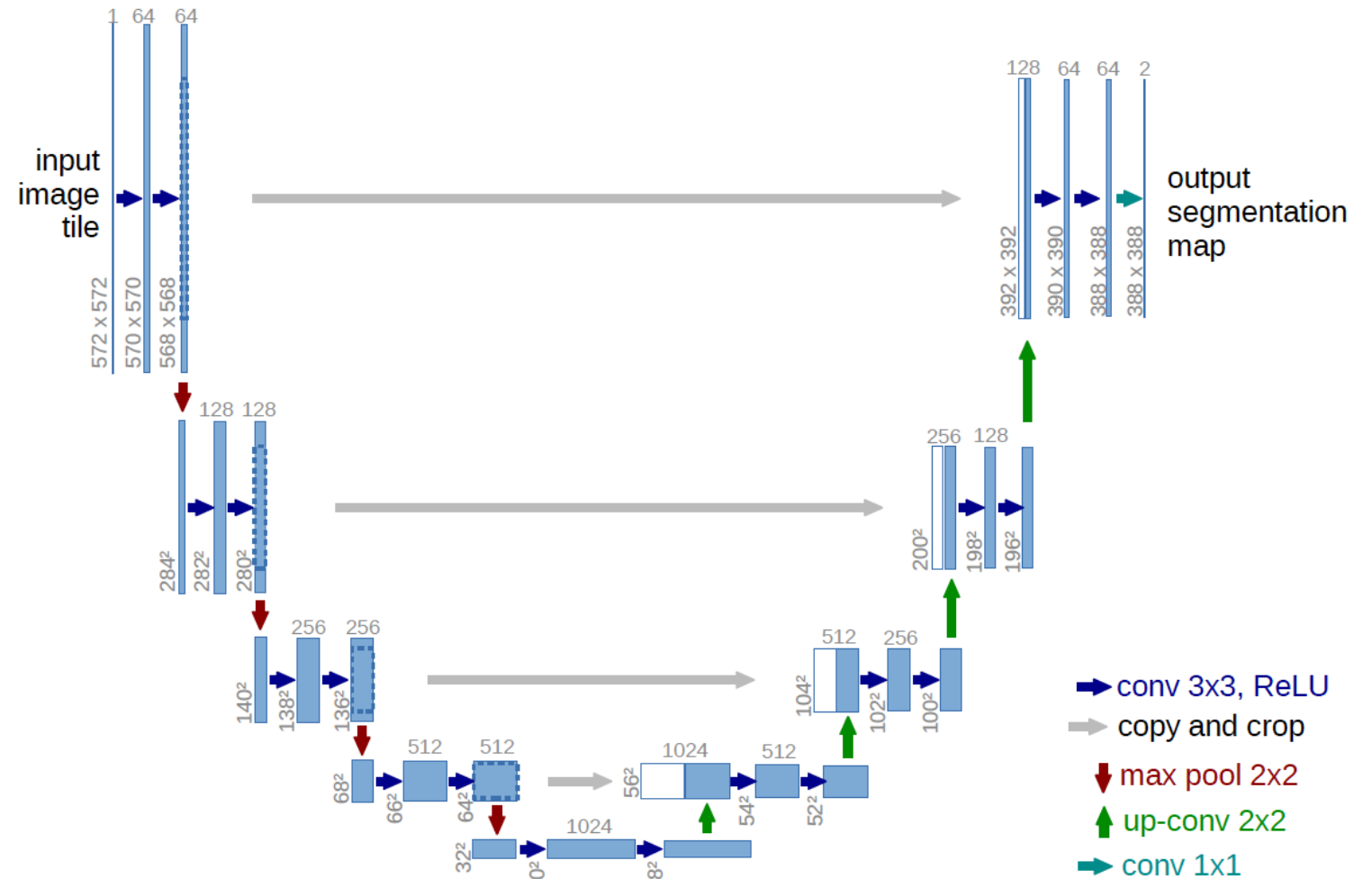
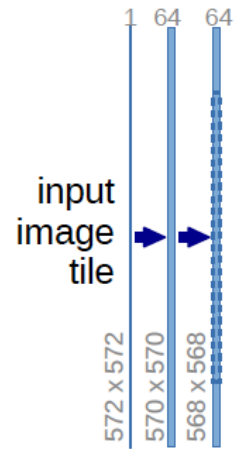


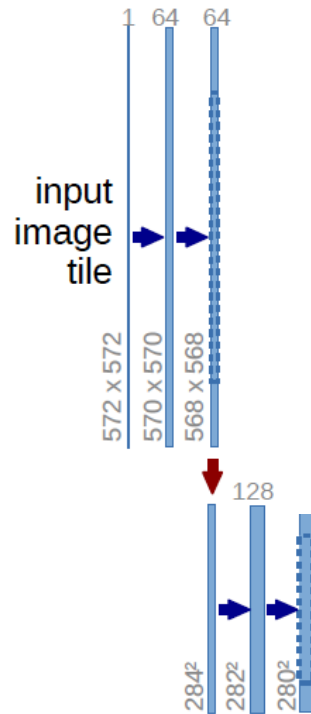
Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

U-net



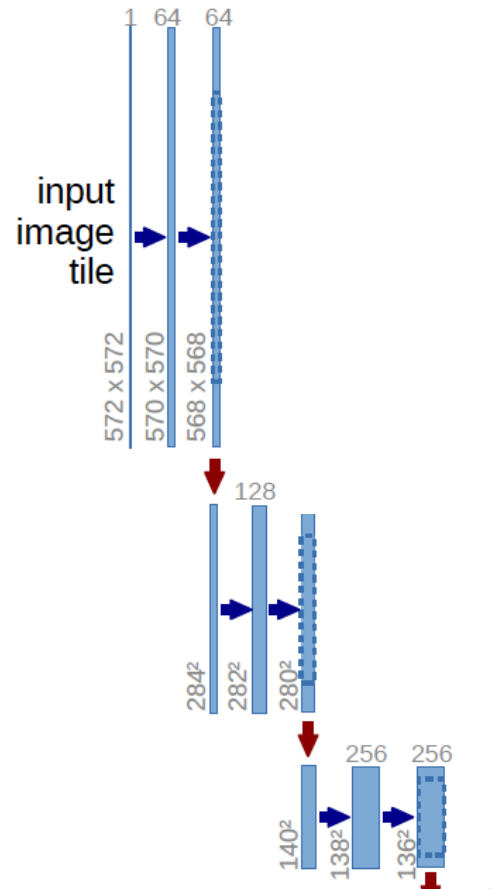
- conv 3x3, ReLU
- copy and crop
- ↓ max pool 2x2
- ↑ up-conv 2x2
- conv 1x1

U-net



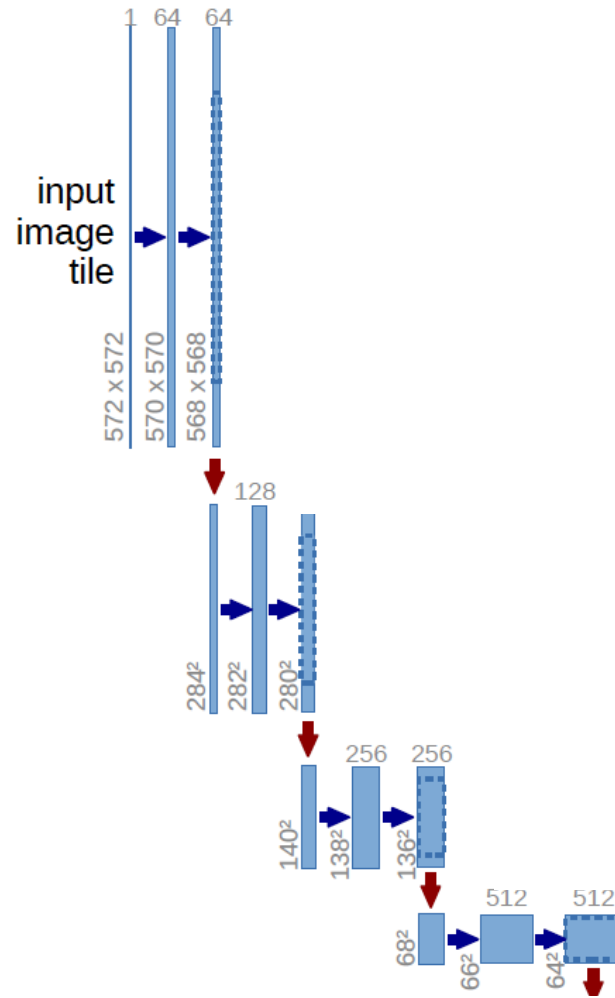
- conv 3x3, ReLU
- copy and crop
- ↓ max pool 2x2
- ↑ up-conv 2x2
- conv 1x1

U-net



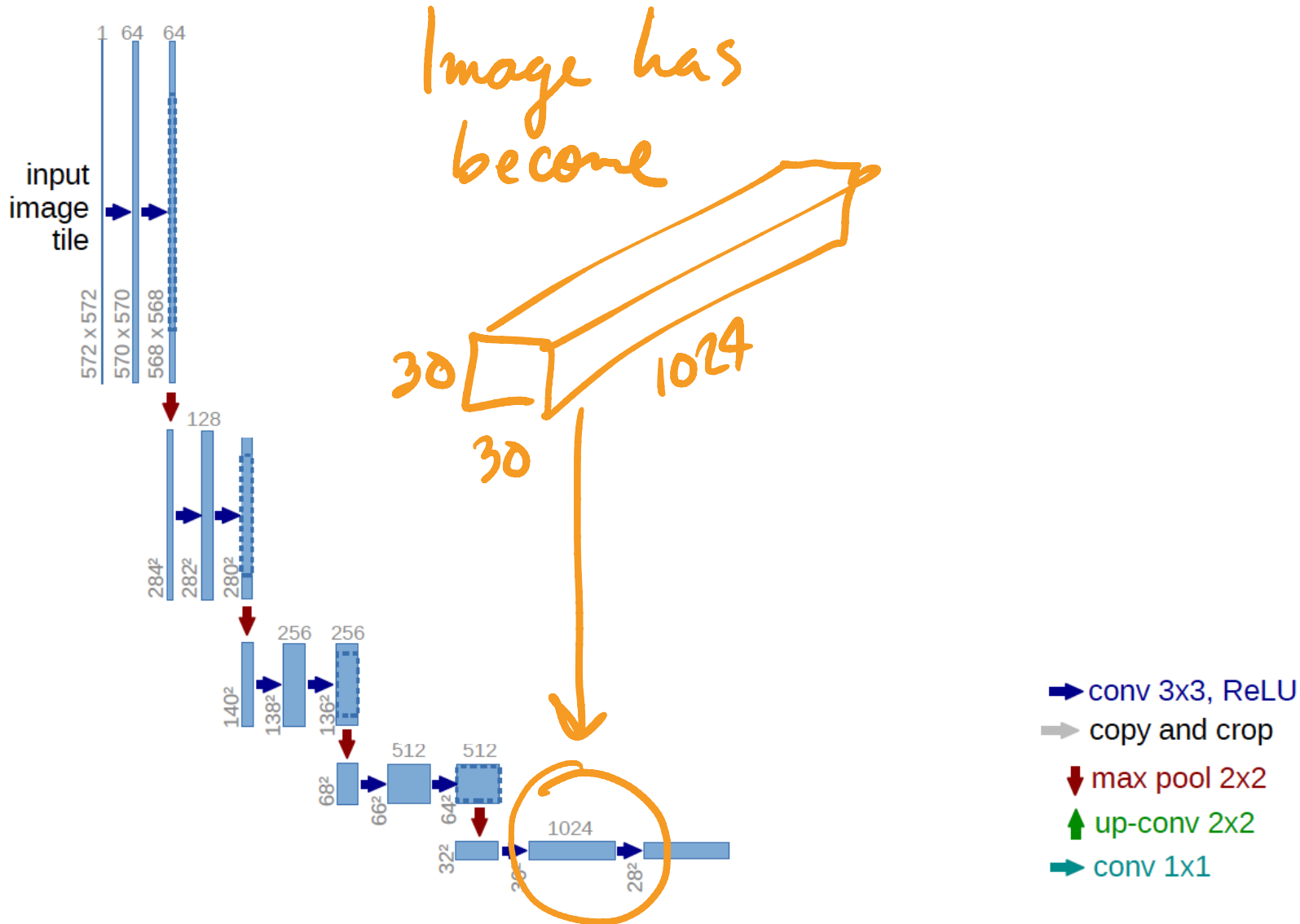
- conv 3x3, ReLU
- copy and crop
- ↓ max pool 2x2
- ↑ up-conv 2x2
- conv 1x1

U-net

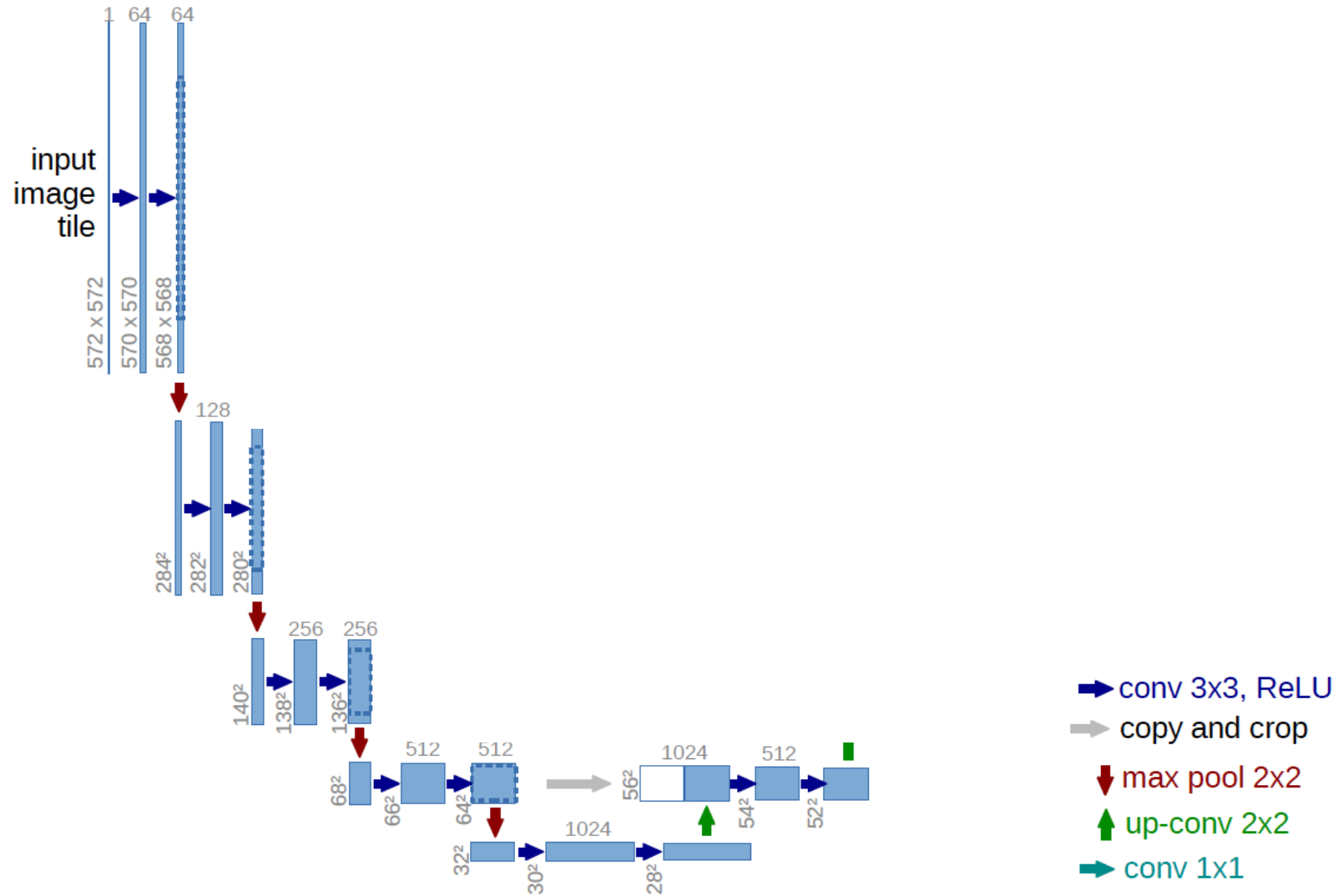


- conv 3x3, ReLU
- copy and crop
- ↓ max pool 2x2
- ↑ up-conv 2x2
- conv 1x1

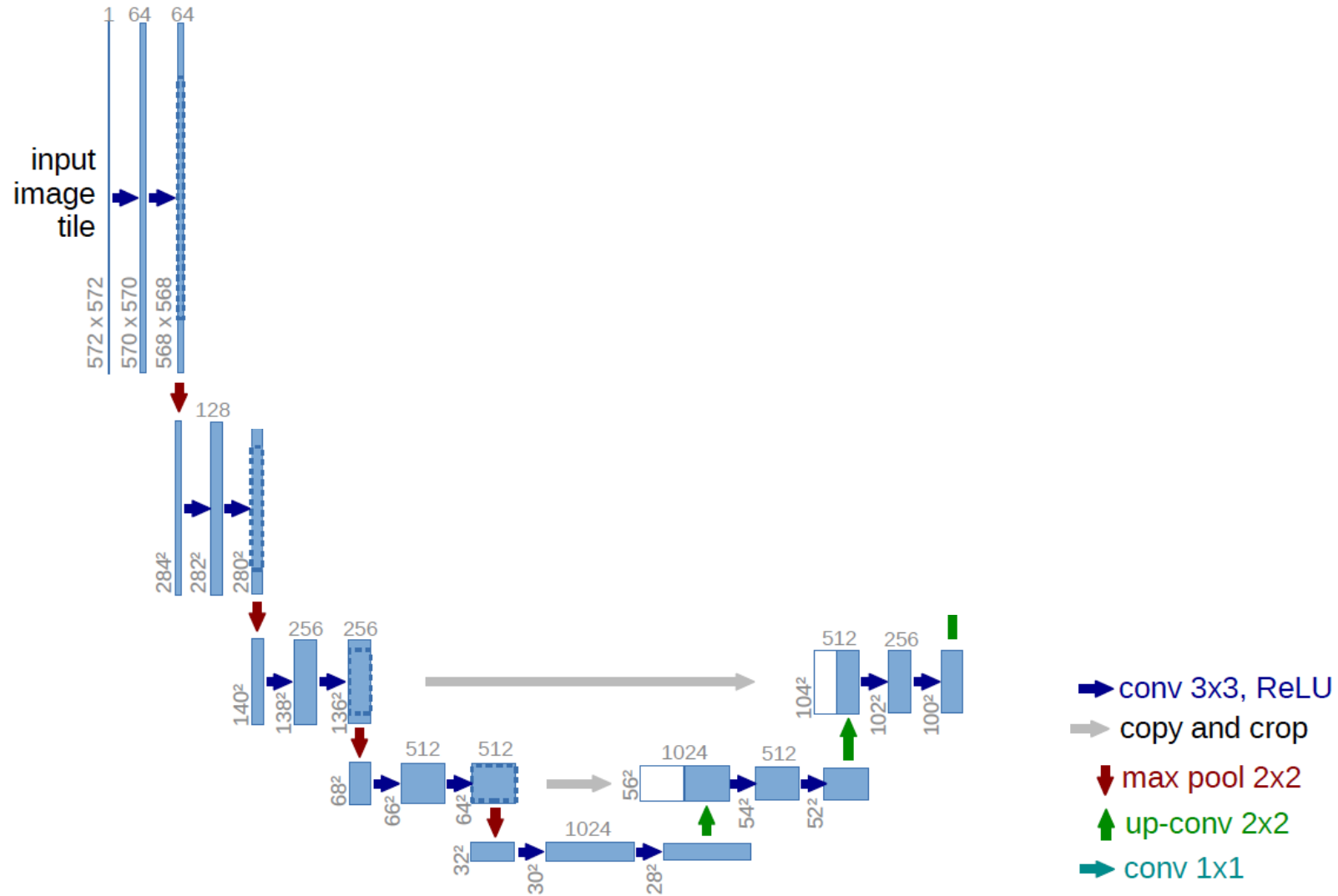
U-net



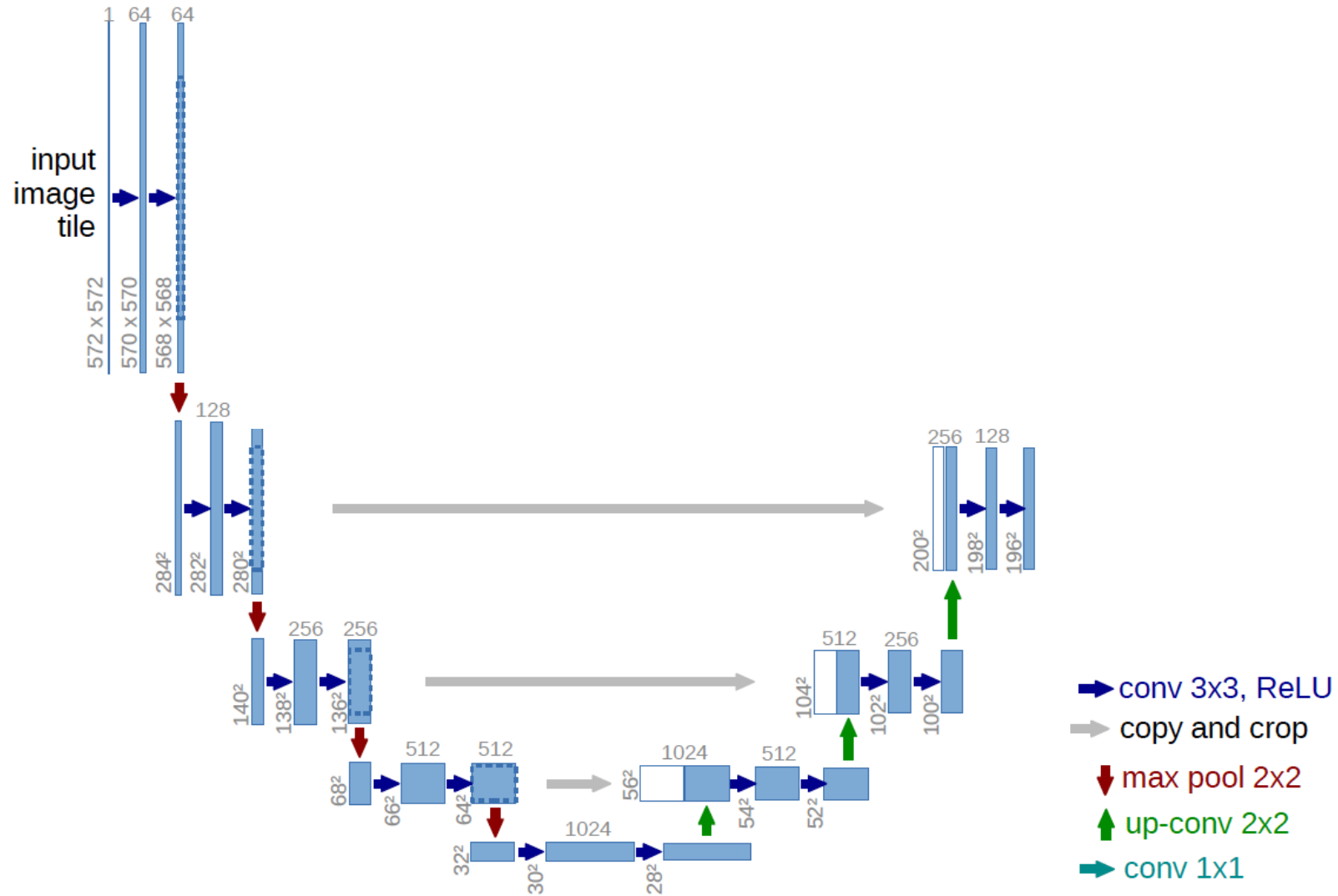
U-net



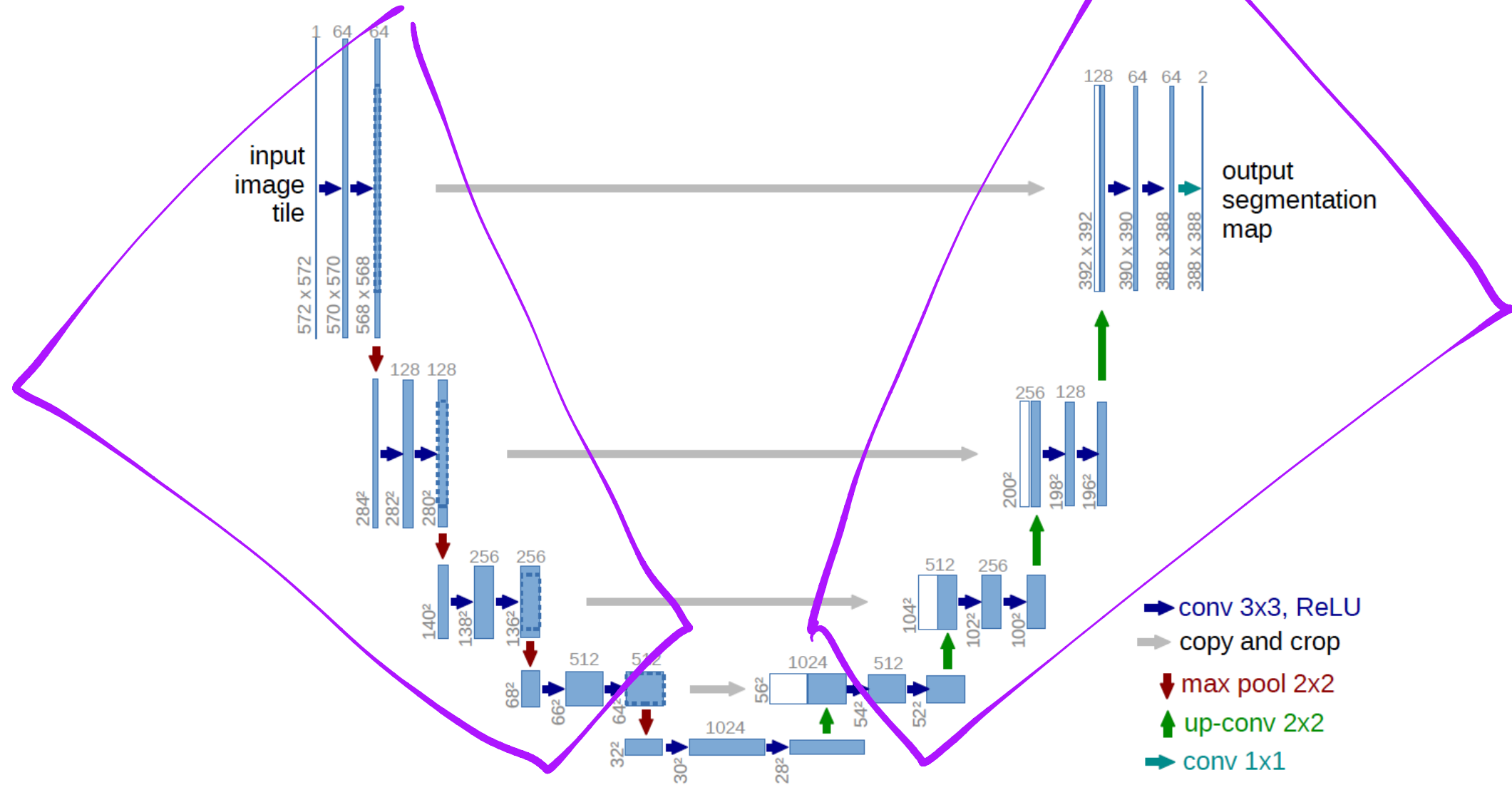
U-net

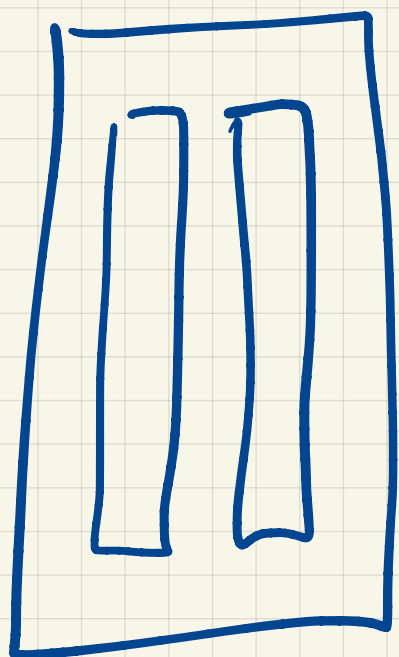


U-net



U-net





U-net

